

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчёт

по лабораторной работе №8

по курсу «Л и О А в ИЗ»

на тему «Обход графа в ширину»

Выполнили студенты группы 24ВВВ4:

Кошелев Р.Д.

Кондратьев С.В.

Приняли к.т.н., доцент:

Юрова О.В.

к.э.н., доцент:

Акифьев И.В.

Пенза 2025

Цель работы: реализовать и изучить обход в ширину (ПОШ/BFS) на графе, представленного матрицей и списками смежности, а также сравнить два варианта очереди — на массиве и на связанном списке.

Общие сведения.

Обход графа в ширину – еще один распространенный способ обхода графов.

Основная идея такого обхода состоит в том, чтобы посещать вершины по уровням удаленности от исходной вершины. Удалённость в данном случае понимается как количество ребер, по которым необходимо прейти до достижения вершины. Например, если для графа на рисунке 1 начать обход из первой вершины, то вершины 3, 6 и 2 будут находиться на уровне удаленности в 1 ребро, а вершины 5 и 4 на уровне удаленности в 2 ребра.

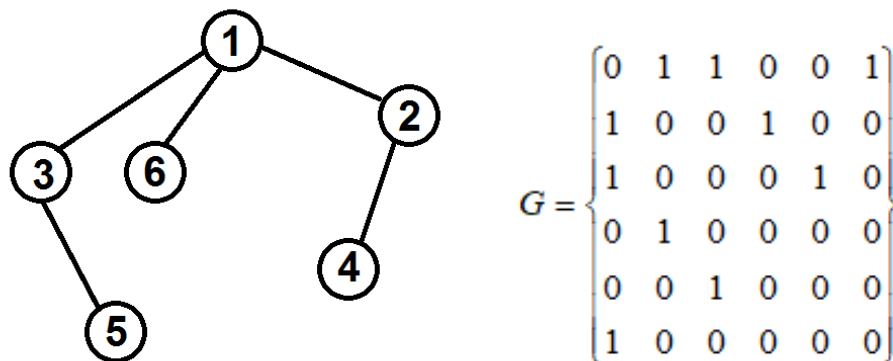


Рисунок 1 – Граф

Тогда при обходе этого графа в ширину, мы сначала посетим вершины первого уровня удаленности (с номерами 2, 3 и 6), и только после того, как закончатся не посещенные вершины на этом уровне, мы перейдем к следующему. На втором уровне мы посетим все вершины, которые удалены от исходной на 2 ребра (вершины 4 и 5).

Так, алгоритм обхода в ширину продолжает осматривать уровень за уровнем, пока не пройдет все доступные вершины.

Чтобы не заходить повторно в уже пройденные вершины, они помечаются, как и в алгоритме обхода в глубину.

Для того, чтобы проход осуществлялся по уровням необходимо хранить информацию о требуемом порядке посещения вершин. Вершины, которые являются ближайшими соседями исходной вершины (из которой начат обход) должны быть посещены раньше, чем соседи соседей и т.д. Такой порядок позволяет задать структура данных «очередь». Просматривая строку матрицы смежности (или список смежности) для текущей вершины мы помещаем всех её ещё не посещенных соседей в очередь. На следующей итерации текущей вершиной становится та, которая стоит в очереди первой и уже её не посещенные соседи будут помещены в очередь. Но место в очереди они займут после тех вершин, которые были помещены туда на предыдущих итерациях.

Таким образом, можно предложить следующую реализацию алгоритма обхода в ширину.

Вход: G – матрица смежности графа.

Выход: номера вершин в порядке их прохождения на экране.

Алгоритм ПОШ

- 1.1. для всех i положим $NUM[i] = \text{False}$ пометим как "не посещенную";
- 1.2. **ПОКА** существует "новая" вершина v
- 1.3. **ВЫПОЛНЯТЬ** BFS (v).

Алгоритм BFS(v):

- 2.1. Создать пустую очередь $Q = \{\}$;
- 2.2. Поместить v в очередь $Q.push(v)$;
- 2.3. пометить v как "посещенную" $NUM[v] = \text{True}$;
- 2.4. **ПОКА** $Q \neq \emptyset$ очередь не пуста **ВЫПОЛНЯТЬ**
- 2.5. $v = Q.front()$ установить текущую вершину;
- 2.6. Удалить первый элемент из очереди $Q.pop()$;

- 2.7. вывести на экран v ;
- 2.8. **ДЛЯ** $i = 1$ **ДО** size_G **ВЫПОЛНЯТЬ**
- 2.9. **ЕСЛИ** $G(v,i) == 1$ **И** $\text{NUM}[i] == \text{False}$
- 2.10. **ТО**
- 2.11. Поместить i в очередь $Q.\text{push}(i)$;
- 2.12. пометить v как "посещенную" $\text{NUM}[v] = \text{True}$;

Реализация состоит из подготовительной части, в которой все вершины помечаются как не посещенные (п.1.1) и осуществляется запуск процедуры обхода для вершин графа (п.1.2, 1.3).

В самой процедуре обхода сначала создается пустая очередь (п. 2.1), в которую помещается исходная вершина, из которой начат обход (п.2.2).

Далее итерационно, пока очередь не опустеет, из нее извлекается первый элемент, который становится текущей вершиной (п. 2.5, 2.6). Затем в цикле просматривается v -я строка матрицы смежности графа $G(v,i)$. Как только алгоритм встречает смежную с v не посещенную вершину (п.2.9), эта вершина помещается в очередь (п.2.11) и помечается как посещенная (п.2.12). После просмотра строки матрицы смежности алгоритм делает следующую итерацию цикла 2.4 или заканчивает работу, если очередь пуста.

Так, если для графа на рисунке 1, мы начнем обход из первой вершины, то на шаге 2.2 она будет помещена в очередь и помечена как посещенная ($\text{NUM}[1] = \text{True}$). Условие цикла 2.4 будет выполнено, вершина 1 будет установлена в качестве текущей (п.2.5) и удалена из очереди (п.2.6). На экран будет выведена единица.

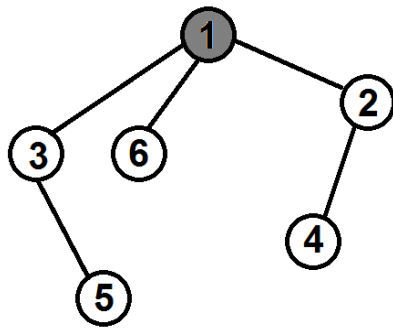


Рисунок 2 – Итерация 1

При просмотре 1-й строки матрицы смежности

$$G = \begin{bmatrix} 0 & \boxed{1} & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

будет найдена смежная вершина с индексом 2 ($G(1,2) = 1$), которая не посещена ($NUM[2] = \text{False}$) и она будет помещена в очередь и помечена ($NUM[2] = \text{True}$). Просмотр строки продолжится, и в очередь будут также помещены и помечены как посещенные вершины с индексами 3 и 6.

К концу первой итерации очередь будет содержать $Q = \{2, 3, 6\}$.

Условие цикла *while* будет выполнено и на следующей итерации вершина 2 будет установлена как текущая и извлечена из очереди, на экран будет выведена двойка.

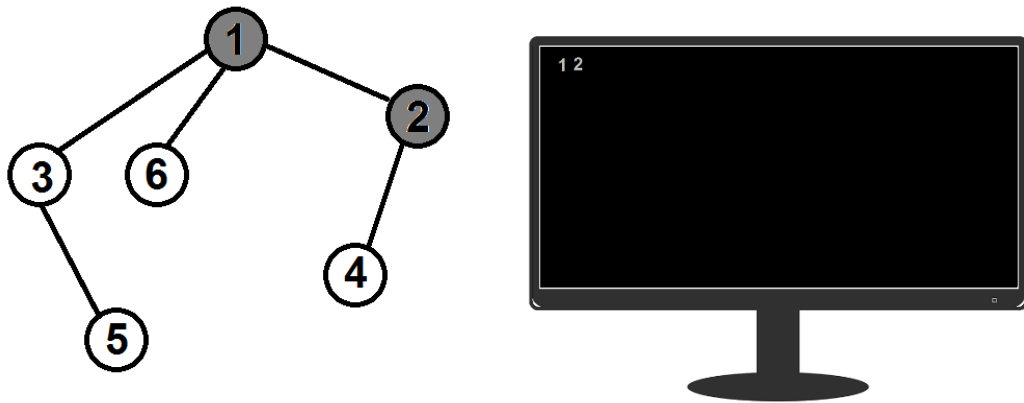


Рисунок 3 – Итерация 2

И алгоритм перейдет к просмотру второй строки матрицы смежности. Первая смежная с вершиной 2 - вершина с индексом 1 ($G(2,1) = 1$),

$$G = \begin{Bmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{Bmatrix}$$

которая к настоящему моменту уже посещена ($NUM[1] == True$), она не будет помещена в очередь. Цикл 2.9 продолжит просмотр матрицы смежности.

$$G = \begin{Bmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{Bmatrix}$$

Следующая найденная вершина, смежная со второй, будет иметь индекс 4 ($G(2,4) = 1$), она не посещена ($NUM[4] == False$) и она будет помещена в очередь и помечена.

К концу второй итерации очередь будет содержать $Q = \{3, 6, 4\}$.

Условие цикла *while* будет выполнено и на следующей итерации вершина 3 будет установлена как текущая и извлечена из очереди, на экран будет выведена тройка.

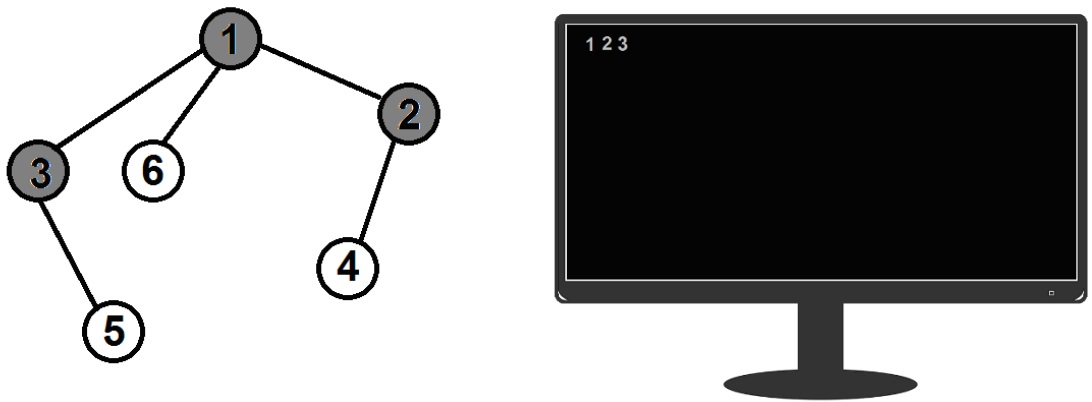


Рисунок 4 – Итерация 3

При просмотре 3-й строки матрицы будет найдена вершина 1, но она уже посещена ($\text{NUM}[1] = \text{True}$), поэтому в очередь помещена не будет.

$$G = \begin{Bmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{Bmatrix}$$

Следующая найденная вершина, смежная с третьей – вершина с номером 5 ($G(3,5) = 1$), она не посещена ($\text{NUM}[5] = \text{False}$) и будет помещена в очередь.

$$G = \begin{Bmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{Bmatrix}$$

К концу третьей итерации очередь будет содержать $Q = \{6, 4, 5\}$.

Условие цикла *while* будет выполнено. На следующей итерации вершина 6 будет установлена как текущая и извлечена из очереди, на экран будет выведена цифра шесть.

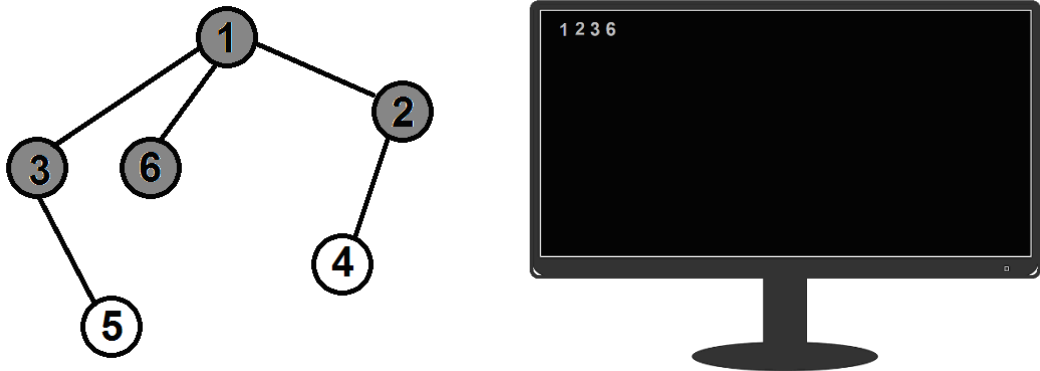


Рисунок 5 – Итерация 4

При просмотре 6-й строки матрицы будет найдена вершина 1, но она уже посещена ($\text{NUM}[1] = \text{True}$), поэтому в очередь помещена не будет.

К концу четвертой итерации очередь будет содержать $Q = \{4, 5\}$.

Далее из очереди будет извлечена вершина 4, установлена как текущая и выведена на экран.

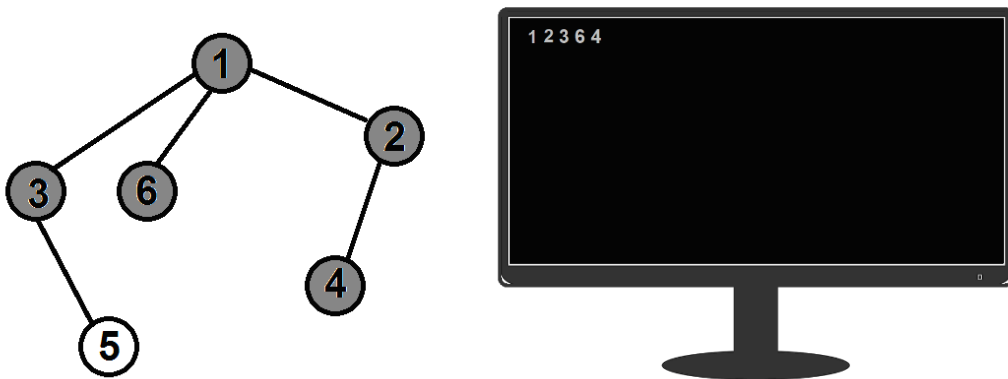


Рисунок 6 – Итерация 5

Так как все вершины кроме пятой уже посещены, то при просмотре 4-й строки в матрице смежности в очередь не будут добавлены вершины.

К концу пятой итерации очередь будет содержать только одну вершину $Q = \{5\}$.

На шестой итерации вершина с номером пять будет установлена как текущая и извлечена из очереди, на экран будет выведена цифра пять.

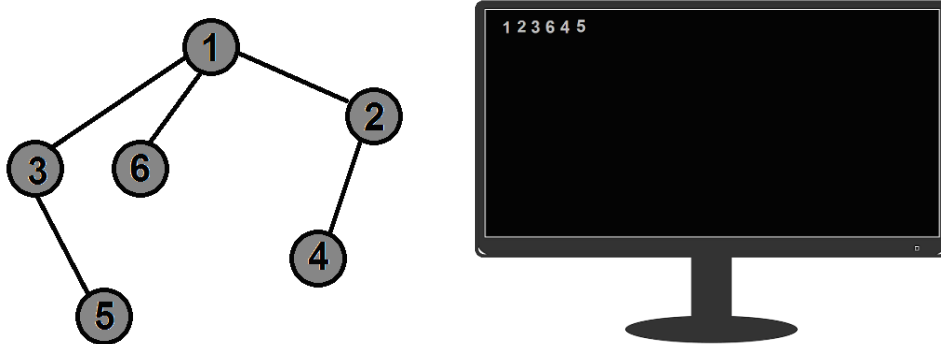


Рисунок 7 – Результат работы обхода

Просмотр 5-й строки матрицы смежности не добавит в очередь новых вершин, так как к этому моменту они уже все помечены как посещенные.

После этой итерации очередь окажется пуста $Q = \{\}$ и алгоритм завершит свою работу.

В конце работы алгоритма все вершины будут посещены. А на экран будут выведены номера вершин в порядке их посещения алгоритмом.

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс `queue` из стандартной библиотеки C++.
- 3.* Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.

Задание 2*

1. Для матричной формы представления графов реализуйте алгоритм обхода в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной в лабораторной работе № 3.

2. Оцените время работы двух реализаций алгоритмов обхода в ширину (использующего стандартный класс `queue` и использующего очередь, реализованную самостоятельно) для графов разных порядков.

Результат работы программы

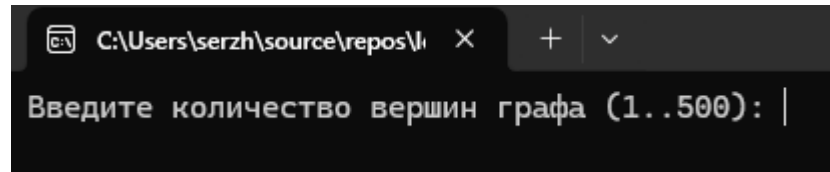


Рисунок 1 – Ввод количества вершин для неориентированного графа

```
Матрица смежности (5 x 5):
0 1 0 1 1
1 0 1 1 1
0 1 0 1 1
1 1 1 0 1
1 1 1 1 0

BFS (матрица + массивная очередь) – порядок обхода вершин:
1 2 4 5 3
Время: 0,000000 сек

BFS (матрица + очередь на списке) – порядок обхода вершин:
1 2 4 5 3
Время: 0,001000 сек

BFS (списки смежности + массивная очередь) – порядок обхода вершин:
1 2 4 5 3
Время: 0,000000 сек

Сравнение времени (матрица): массивная очередь = 0,000000 с, очередь на списке = 0,001000 с
```

Рисунок 2 – Вывод матрицы смежности, порядок и время обхода вершин

Вывод: реализованы генерация матрицы, построение списков смежности и три варианта BFS; измерения показали, что очередь на массиве проще и быстрее для небольших графов, а очередь на списке более универсальна и устойчива при работе с динамическими или большими структурами.

Приложение А

Листинг

```
#include <locale.h>

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#include <string.h>

#include <ctype.h>


#define MAX_VERTICES 500


//Вспомогательные функции для проверки ввода


int read_positive_int(const char* prompt, int min, int max) {
    char buffer[256];
    long val;
    char* endptr;
    while (1) {
        printf("%s", prompt);
        if (!fgets(buffer, sizeof(buffer), stdin)) {
            // ввод закончился
            printf("Ошибка ввода. Попробуйте снова.\n");
            continue;
        }
        // Убрать перевод строки
        buffer[strcspn(buffer, "\n")] = '\0';
```

```

// Проверка на пустую строку
if (buffer[0] == '\0') {
    printf("Пустой ввод. Введите целое положительное число.\n");
    continue;
}

// Проверка на то, что все символы - цифры (плюс возможный ведущий '+')
int i = 0;
if (buffer[0] == '+') i = 1;
int ok = 1;
for (; buffer[i]; ++i) {
    if (!isdigit((unsigned char)buffer[i])) { ok = 0; break; }
}
if (!ok) {
    printf("Разрешены только целые положительные числа. Попробуйте снова.\n");
    continue;
}

val = strtol(buffer, &endptr, 10);
if (endptr == buffer) {
    printf("Неверный ввод. Попробуйте снова.\n");
    continue;
}
if (val < min || val > max) {
    printf("Число должно быть в диапазоне %d..%d. Попробуйте снова.\n",
min, max);
    continue;
}

```

```

    }
    return (int)val;
}
}

```

//Очередь на массиве (простая реализация)

```

typedef struct {
    int* data;
    int front;
    int rear;
    int capacity;
} ArrayQueue;

```

```

void aq_init(ArrayQueue* q, int capacity) {
    q->data = (int*)malloc(sizeof(int) * capacity);
    q->front = 0;
    q->rear = -1;
    q->capacity = capacity;
}

```

```

void aq_free(ArrayQueue* q) {
    free(q->data);
}

```

```

int aq_is_empty(ArrayQueue* q) {
    return q->rear < q->front;
}

```

```
}
```

```
void aq_enqueue(ArrayQueue* q, int v) {  
    if (q->rear + 1 >= q->capacity) {  
        return;  
    }  
    q->data[++q->rear] = v;  
}
```

```
int aq_dequeue(ArrayQueue* q) {  
    if (aq_is_empty(q)) return -1;  
    return q->data[q->front++];  
}
```

//Очередь на связанном списке (самостоятельная реализация)

```
typedef struct Node {  
    int val;  
    struct Node* next;  
} Node;
```

```
typedef struct {  
    Node* front;  
    Node* rear;  
} ListQueue;
```

```
void lq_init(ListQueue* q) {
```

```
    q->front = q->rear = NULL;
}
```

```
int lq_is_empty(ListQueue* q) {
    return q->front == NULL;
}
```

```
void lq_enqueue(ListQueue* q, int v) {
    Node* p = (Node*)malloc(sizeof(Node));
    if (!p) { perror("malloc"); exit(EXIT_FAILURE); }
    p->val = v; p->next = NULL;
    if (q->rear == NULL) {
        q->front = q->rear = p;
    }
    else {
        q->rear->next = p;
        q->rear = p;
    }
}
```

```
int lq_dequeue(ListQueue* q) {
    if (lq_is_empty(q)) return -1;
    Node* p = q->front;
    int v = p->val;
    q->front = p->next;
    if (q->front == NULL) q->rear = NULL;
    free(p);
}
```

```
    return v;
}
```

```
void lq_free_all(ListQueue* q) {
    while (!lq_is_empty(q)) lq_dequeue(q);
}
```

//Генерация и печать матрицы смежности

```
void generate_random_adj_matrix(int n, int** matrix) {
    int density = 40 + rand() % 41;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) matrix[i][j] = 0;
    }
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            int r = rand() % 100;
            if (r < density) {
                matrix[i][j] = matrix[j][i] = 1;
            }
        }
    }
}
```

```
void print_matrix(int n, int** matrix) {
    printf("Матрица смежности (%d x %d):\n", n, n);
    for (int i = 0; i < n; ++i) {
```

```

        for (int j = 0; j < n; ++j) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

```

//Построение списков смежности из матрицы

```

int** build_adj_list(int n, int** matrix, int* deg) {
    // deg[i] = степень вершины i
    for (int i = 0; i < n; ++i) {
        deg[i] = 0;
        for (int j = 0; j < n; ++j)
            if (matrix[i][j]) deg[i]++;
    }
    int** adj = (int**)malloc(sizeof(int*) * n);
    for (int i = 0; i < n; ++i) {
        if (deg[i] > 0)
            adj[i] = (int*)malloc(sizeof(int) * deg[i]);
        else
            adj[i] = NULL;
        int k = 0;
        for (int j = 0; j < n; ++j)
            if (matrix[i][j]) adj[i][k++] = j;
    }
    return adj;
}

```

```
}
```

```
void free_adj_list(int n, int** adj) {  
    for (int i = 0; i < n; ++i) if (adj[i]) free(adj[i]);  
    free(adj);  
}
```

//BFS: матрица + массивная очередь

```
void BFS_matrix_arrayqueue(int n, int** G, int start, int* visited) {  
    ArrayQueue q;  
    aq_init(&q, n);  
    aq_enqueue(&q, start);  
    visited[start] = 1;  
  
    while (!aq_is_empty(&q)) {  
        int v = aq_dequeue(&q);  
        printf("%d ", v + 1); // выводим вершины с 1  
        for (int i = 0; i < n; ++i) {  
            if (G[v][i] == 1 && visited[i] == 0) {  
                aq_enqueue(&q, i);  
                visited[i] = 1; // пометить i как посещенную  
            }  
        }  
    }  
  
    aq_free(&q);
```

```
}
```

```
// BFS: матрица + очередь на списке
```

```
void BFS_matrix_listqueue(int n, int** G, int start, int* visited) {
```

```
    ListQueue q;
```

```
    lq_init(&q);
```

```
    lq_enqueue(&q, start);
```

```
    visited[start] = 1;
```

```
    while (!lq_is_empty(&q)) {
```

```
        int v = lq_dequeue(&q);
```

```
        printf("%d ", v + 1);
```

```
        for (int i = 0; i < n; ++i) {
```

```
            if (G[v][i] == 1 && visited[i] == 0) {
```

```
                lq_enqueue(&q, i);
```

```
                visited[i] = 1;
```

```
            }
```

```
        }
```

```
    }
```

```
    lq_free_all(&q);
```

```
}
```

```
//BFS: списки смежности + массивная очередь
```

```
void BFS_adjlist_arrayqueue(int n, int** adj, int* deg, int start, int* visited) {
```

```

ArrayQueue q;
aq_init(&q, n);
aq_enqueue(&q, start);
visited[start] = 1;

while (!aq_is_empty(&q)) {
    int v = aq_dequeue(&q);
    printf("%d ", v + 1);
    for (int k = 0; k < deg[v]; ++k) {
        int to = adj[v][k];
        if (!visited[to]) {
            aq_enqueue(&q, to);
            visited[to] = 1;
        }
    }
}
aq_free(&q);
}

// Алгоритм ПОШ: обойти все компоненты, вызывая BFS

void POSH_matrix_arrayqueue_all(int n, int** G) {
    int* visited = (int*)calloc(n, sizeof(int));
    for (int v = 0; v < n; ++v) {
        if (!visited[v]) {
            BFS_matrix_arrayqueue(n, G, v, visited);
        }
    }
}

```

```
    }  
    free(visited);  
}
```

```
void POSH_matrix_listqueue_all(int n, int** G) {  
    int* visited = (int*)calloc(n, sizeof(int));  
    for (int v = 0; v < n; ++v) {  
        if (!visited[v]) {  
            BFS_matrix_listqueue(n, G, v, visited);  
        }  
    }  
    free(visited);  
}
```

```
void POSH_adjlist_arrayqueue_all(int n, int** adj, int* deg) {  
    int* visited = (int*)calloc(n, sizeof(int));  
    for (int v = 0; v < n; ++v) {  
        if (!visited[v]) {  
            BFS_adjlist_arrayqueue(n, adj, deg, v, visited);  
        }  
    }  
    free(visited);  
}
```

```
int main(void) {  
  
    setlocale(LC_ALL, "rus");
```

```
srand((unsigned)time(NULL));
```

```
int n = read_positive_int("Введите количество вершин графа (1..500): ", 1, MAX_VERTICES);
```

```
// Выделим матрицу как массив указателей на строки
```

```
int** G = (int**)malloc(sizeof(int*) * n);
```

```
for (int i = 0; i < n; ++i) {
```

```
    G[i] = (int*)malloc(sizeof(int) * n);
```

```
}
```

```
generate_random_adj_matrix(n, G);
```

```
print_matrix(n, G);
```

```
// Построим списки смежности
```

```
int* deg = (int*)malloc(sizeof(int) * n);
```

```
int** adj = build_adj_list(n, G, deg);
```

```
printf("\nBFS (матрица + массивная очередь) — порядок обхода вершин:\n");
```

```
clock_t t1 = clock();
```

```
POSH_matrix_arrayqueue_all(n, G);
```

```
clock_t t2 = clock();
```

```
double time_array_queue = (double)(t2 - t1) / CLOCKS_PER_SEC;
```

```
printf("\nВремя: %.6f сек\n", time_array_queue);
```

```
printf("\nBFS (матрица + очередь на списке) — порядок обхода вершин:\n");
```

```

t1 = clock();
POSH_matrix_listqueue_all(n, G);
t2 = clock();
double time_list_queue = (double)(t2 - t1) / CLOCKS_PER_SEC;
printf("\nВремя: %.6f сек\n", time_list_queue);

printf("\nBFS (списки смежности + массивная очередь) — порядок обхода
вершин:\n");
t1 = clock();
POSH_adjlist_arrayqueue_all(n, adj, deg);
t2 = clock();
double time_adjlist = (double)(t2 - t1) / CLOCKS_PER_SEC;
printf("\nВремя: %.6f сек\n", time_adjlist);

printf("\nСравнение времени (матрица): массивная очередь = %.6f с, очередь
на списке = %.6f с\n",
    time_array_queue, time_list_queue);

// Очистка
for (int i = 0; i < n; ++i) free(G[i]);
free(G);
free_adj_list(n, adj);
free(deg);

return 0;
}

```